→ Based on PMI

$$\Sigma n = \frac{n(n+1)}{2}$$

Task1: $f(0)$, $f(1)$ is true

Assume $f(K)$ is true

Task2: Prove $f(K+1)$ is true

$$\Sigma K = \frac{K(K+1)}{2} \quad \Big| \quad \Sigma K+1 = \frac{(K+1)(K+2)}{2}$$

$$\Sigma K+1 = 2K + K+1$$

$$= \frac{K(K+1)}{2} + \frac{2(K+1)}{2}$$

$$= \left(\frac{K+1}{2}\right)(K+2)$$

- What is Base Case?
- What is my hypothesis?
- What is my induction step?

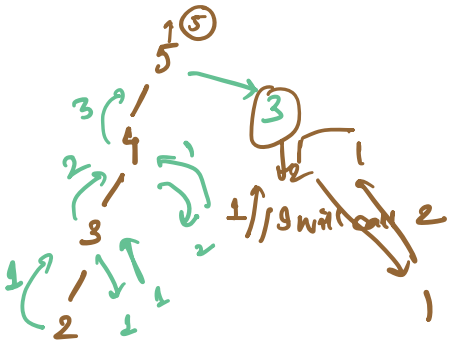
Problem Solving

$$\Sigma n = n + \Sigma n - 1$$

```
def sum_n(n):
    if n == 0:
        return 0
    small Output = sum_n(n-1)
    Output = smallOutput + n
    return output.
```

Fibonacci Number

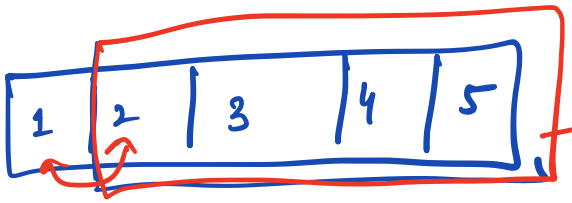
```
def fib(n):
    if n==1 or n==2:
        return 1
    fib_1 = fib(n-1)
    fib_2 = fib(n-2)
    output = fib_1 + fib_2
    return output.
```



Recursion Depth

```
import sys
sys.setrecursionlimit(3000)
```

List is sorted or not



① If returns sorted.
② Compare 1st & II

Base Case: list is of size 0 or 1.

I.H : list of size l-1, my function works

I.S : ① check [0] > [1]
return false
else
check for smaller list starting from [1 . .].

```
def isSorted (a):
    l = len (a)
    if l == 0 or l == 1:
        return True
    if a[0] > a[1]:
        return False

    smallerlist = a[1:]
    is smallerlist sorted = is Sorted (smallerlist)
    if is Smaller List Sorted
        True
    else
        return false
```

```
b = a[1:]
```

```
def isfound (arr, x)
    ( ———— h, x)
    if len(n) = 0
        return false
    if a[0] = x
        return True
    found = a[1:]
    isfound = is found (found)
    if ~~sma~~ is found
        True
    else
```

```
def isSortedBetter (a, si):
    l = len(a)
    if si == l-1 or si == l:
        return True
    if a[si] > a[si+1]:
        return false
    isSmallerPart sorted = isSortedBetter (a, si+1)
    return isSmallerPart Sorted.
```

Find index of x

1) Base Case → ① if len(arr) == 0 then → -1.

2) Recursive Call

3) Small Calculation → ② if a[0] == x then → return 0

→ ③ (a+1, size-1, x) = ans

a+1 ↓ arr

size-1 ↓ len

Return a+1

if ans == -1

return -1

crossed

si == l.

```
def firstIndexB (a, x, si):
    l = len(a)
    if si == l:
        return -1
    if a[si] == x:
        return si
    smallerListOutput = firstIndexB(a, x, si+1)
    return smallerListOutput
```

print (firstIndexB(a,7,0))

replacePi (

① len = 0
return

② small_output (arr, pi, s+1)

~~str[s][s]~~

str [s, s+1] = pi

return "3.14" +

replacePi

[str[2:0]

else

return

aabbccd

↓

s[0] = a

↳ check s[1] = a

remove ⇒ s[1]

else remove_char = s[1]

Binary Search → list must be sorted.

Merge Sort

```
def merge_sort (a):
    mid = len(a) //2
    a1 = a[0: mid]
    a2 = a[mid:]
    merge_sort (a1)
    merge_sort (a2)
    merge (a1, a2, a)
```

if len(a) == 0 || len(a) == 1!
return.

Induction Hypothesis

```
def merge (a1, a2, a)
    i = 0
    j = 0
    while i < len a1 and j < len (a2):
        if (a1[i] < a2[j]):
            a[k] = a1[i]
            k = k+1
            i = i+1
        else
            a[k] = a2[j]
            k = k+1
            j = j+1
    while i < len (a1):
        a[k] = a1[i]
        k = k+1
        i = i+1
    while j < len (a2):
        a[k] = a2[j]
        k = k+1
        j = j+1
```